System and Devices (SYS 3)

Lecture 9: ILP – Dynamic Scheduling

Dr Pengcheng Liu

COM00018I

Dynamic Scheduling

- Rearrange execution order of instructions to reduce stalls while maintaining data flow
- Advantages:

  - Compiler doesn't need to have knowledge of micro-architecture

  - Handles cases where dependencies are unknow at compile time
- Disadvantages:

  - Substantial increase in hardware complexity

  - Complicates exceptions

```
Add  R1, R2, R3
Sub  R6, R1, R5

Str  R2, 8(R1)
Ld   R6, 24(R3)

Str  R2, 0(R1)
Ld   R8, 0(R1)
```

How Dynamic Scheduling Works?

Limitation of simple pipelining

- In-order instruction issue and execution
- Instructions are issued in program order
- If an instruction is stalled in the pipeline, no later instructions can proceed

1 Add R1, R2, R3

2 Sub R4, R1, R3

3 And R6, R1, R7

4 Or R8, R1, R9

How Dynamic Scheduling Works?

Limitation of simple pipelining

- In-order instruction issue and execution
- Instructions are issued in program order
- If an instruction is stalled in the pipeline, no later instructions can proceed

If instruction j depends on a long-running instruction i, currently in execution in the pipeline, then all instructions after j must be stalled until i is finished and j can execute

```
DIV.D      F0,F2,F4
ADD.D      F10,F0,F8
SUB.D      F12,F8,F14
```

TEF Gold Teaching Excellence Framework

How Dynamic Scheduling Works?

- Separate the issue process into two parts:
  - Checking for any structural hazards
  - Waiting for the absence of a data hazard
- Use in-order instruction issue but we want an instruction to begun execution as soon as its data operands are available.
- Out-of-Order execution → out-of-order completion.
- Out-of-Order execution introduces the possibility of WAR and WAW hazards

```
DIV.D     F0,F2,F4
ADD.D     F10,F0,F8
SUB.D     F12,F8,F14
```

WAR

Add R2, R1, R0

Sub R0, R5, R6

WAW

Add R2, R1, R0

Sub R2, R5, R6

How Dynamic Scheduling Works?

- WAR and WAW hazards – solved by register renaming
- Possibility of imprecise exception (2 possibilities)

  - The pipeline may have already completed instructions that are later in program order than the instruction causing the exception

  - The pipeline may have not yet completed some instructions that are earlier in program order than the instruction causing the exception

```
1    DIV.D     F0,F2,F4
2    ADD.D     F6,F0,F8
3    SUB.D     F8,F10,F14
4    MUL.D     F6,F10,F8
```

How Dynamic Scheduling Works?

- To allow out-of-order execution, split the ID stage into two
  - **Issue**: Decode instructions, check for structural hazards
  - **Read operands**: Wait until no data hazards, then read operands
- In a dynamically scheduled pipeline, all instructions pass through the **issue stage in order** (in-order issue); however, they can be stalled or bypass each other in the second stage (read operands) and thus enter execution out of order.
- Done by **score boarding technique**
- Approach used: **Tomasulo's algorithm**

Register Renaming

Name dependence with F6

```
DIV.D    F0,F2,F4
ADD.D    F6,F0,F8
S.D      F6,0(R1)
SUB.D    F8,F10,F14
MUL.D    F6,F10,F8
```

RAW hazard on T

```
DIV.D    F0,F2,F4
ADD.D    S,F0,F8
S.D      S,0(R1)
SUB.D    T,F10,F14
MUL.D    F6,F10,T
```

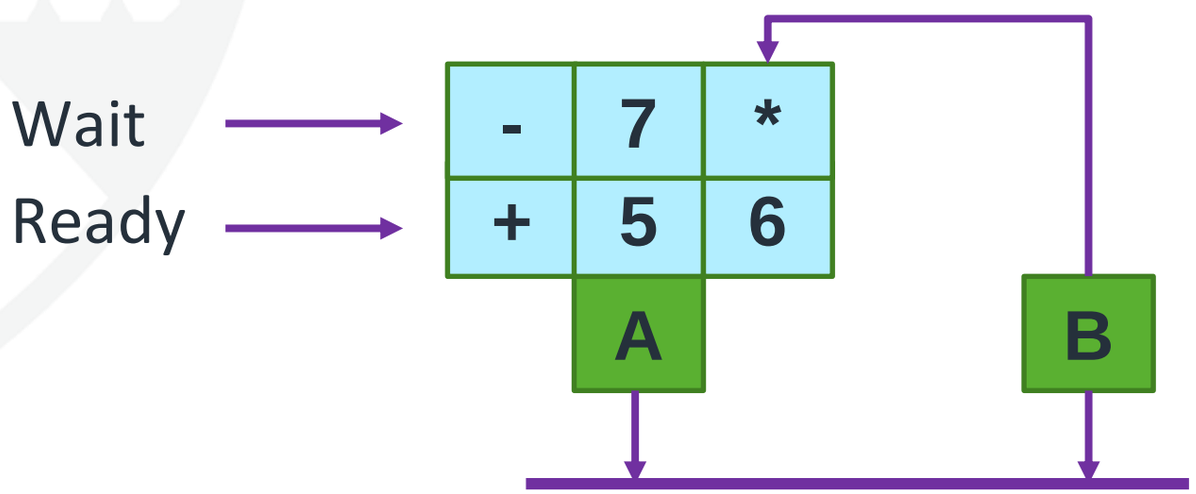
Register Renaming

- Register renaming is done by reservation stations (RS)
- Each RS contains:

  - The instruction (operation to be done)

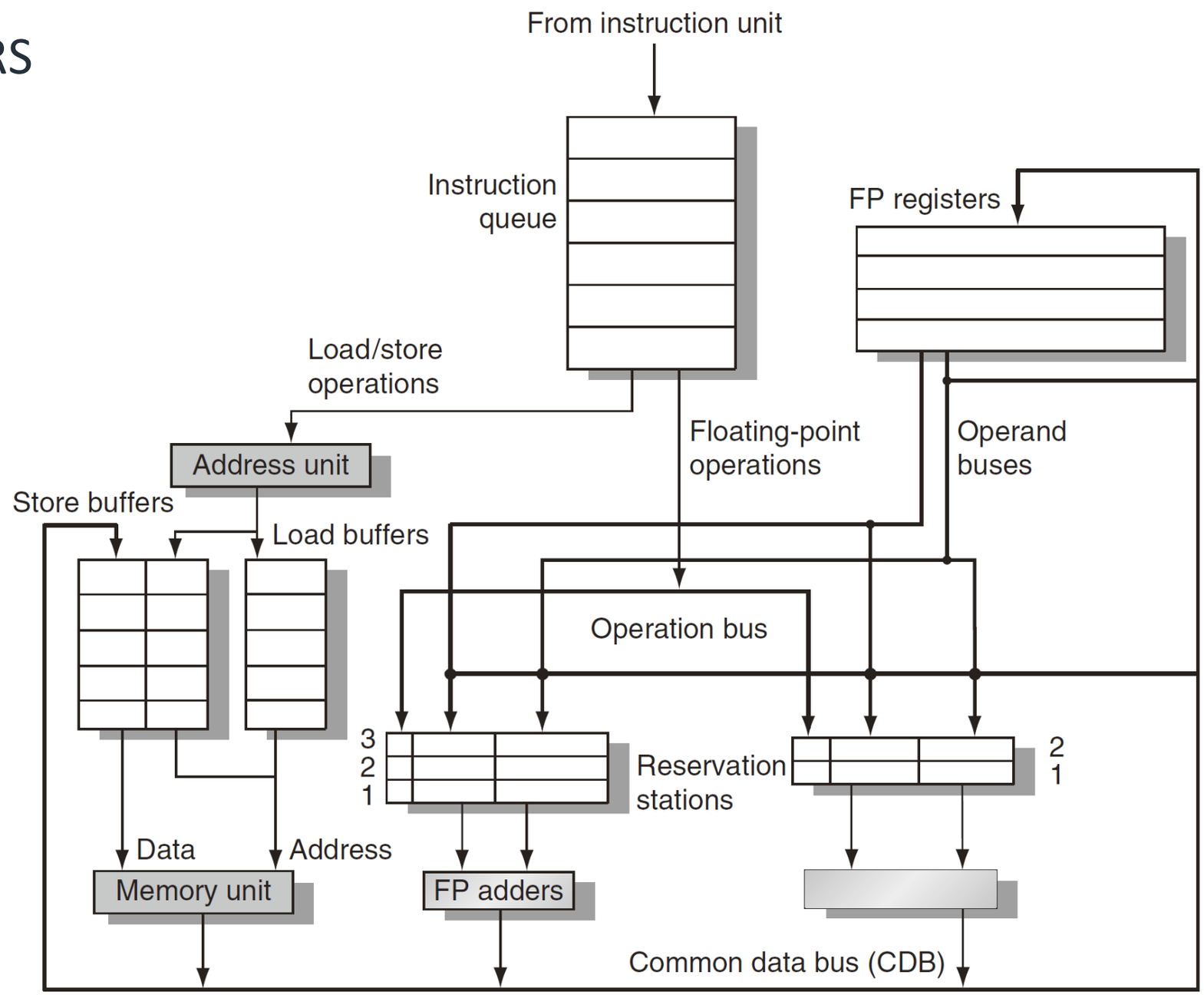
  - Buffered operand values (when available)

  - Reservation station number of instruction providing the operand values
- RS fetches and buffers an operand as soon as it becomes available (not necessarily involving register file)
- Pending instructions designate the RS that will provide input
- Result values broadcast on common data bus (CDB)



Tomasulo's Algorithm

- Load and store buffers contain data and addresses
- They also act like RS



The basic structure of a MIPS floating-point unit using Tomasulo's algorithm. Instructions are sent from the instruction unit into the instruction queue from which they are issued in first-in, first-out (FIFO) order. The reservation stations include the operation and the actual operands, as well as information used for detecting and resolving hazards.

Dynamic Scheduling - Tomasulo

- Issue

  - Get next instruction from FIFO queue

  - If RS available, issue the instruction to the RS with operand values if available

  - If operand values not available, stall the instruction

Add R2, R3, R1

Dynamic Scheduling - Tomasulo

- Execute

  - When operand becomes available, store it in any reservation stations waiting for it

  - When all operands are ready, execute the instruction

  - Loads and store uses buffers

  - No instruction will initiate execution until all branches that precede it in program order have completed

Dynamic Scheduling - Tomasulo

- Write result

  - Write result into CDB (there by it reaches the reservation station, store buffer and register file) with name of execution unit that generated the result

  - Stores must wait until address and value are received

Summary

Suggested Readings

Computer Architecture: A Quantitative Approach (5th Edition)
by John Hennessy and David A Patterson:

- pp. 167 - 176: Sections 3.4